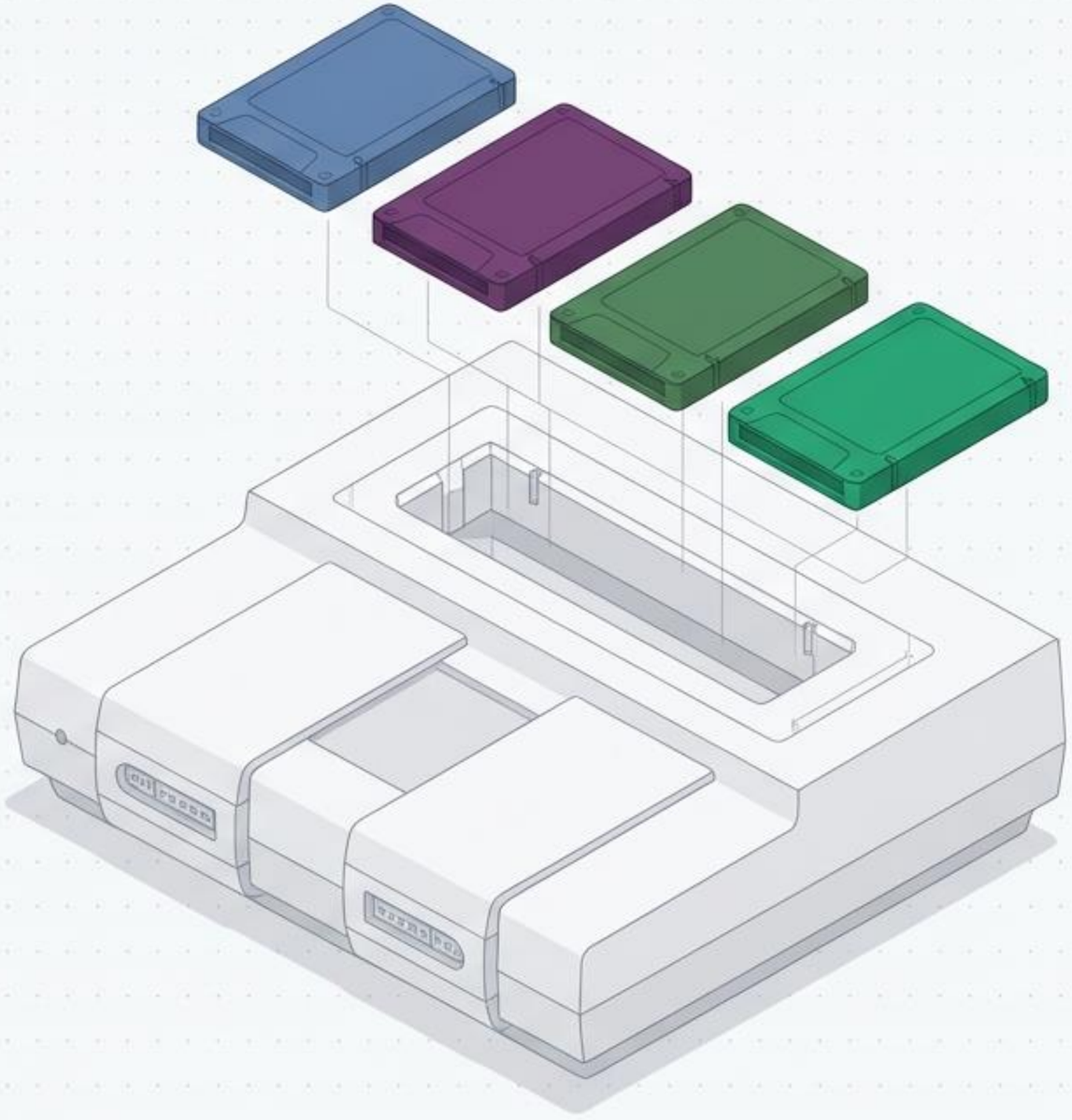
# Data Cartridges for Frontend Developers

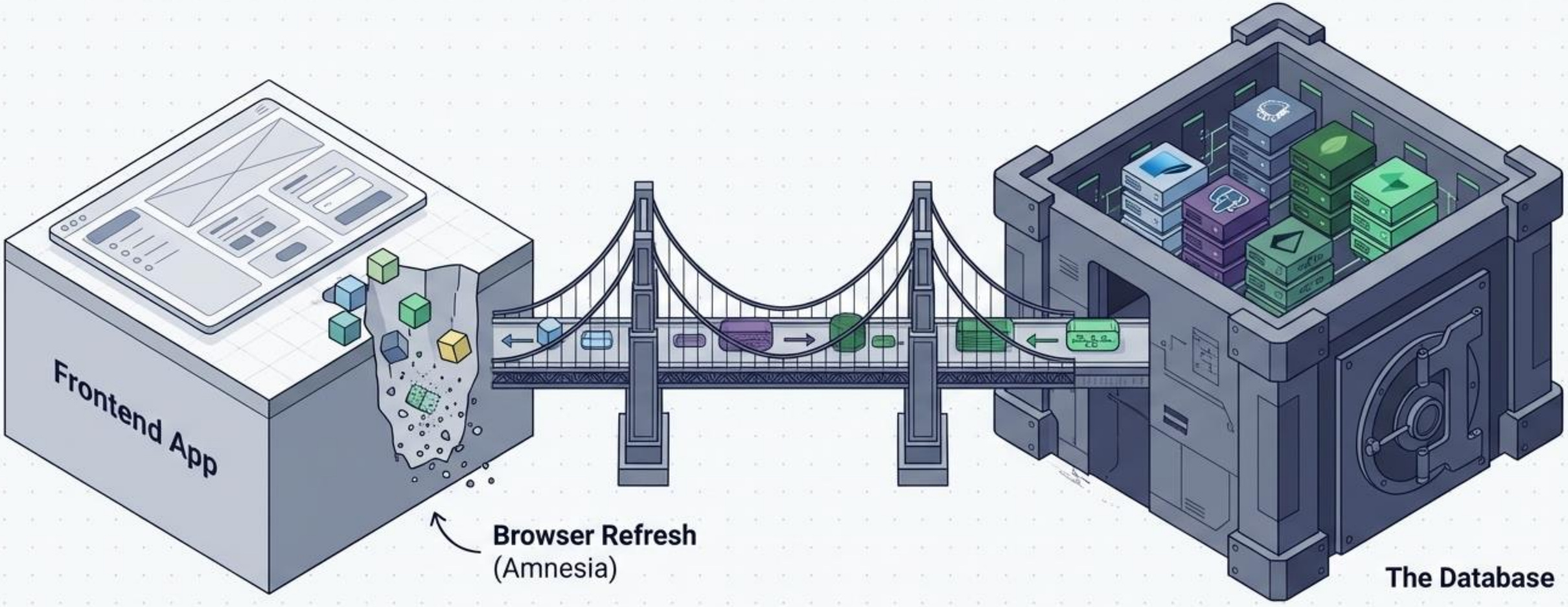
## Integrating Databases into JavaScript Apps with Astro

A visual guide to SQL, NoSQL, and Realtime architectures.



NotebookLM

# Why Frontend Developers Need Databases



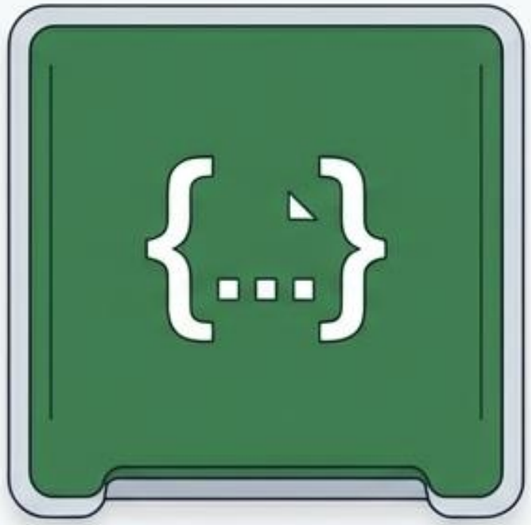

→ To cure application amnesia and provide permanent, queryable state.

NotebookLM

# Three Database Paradigms

**SQL**

Relational & Structured



**NoSQL**

Document & Flexible

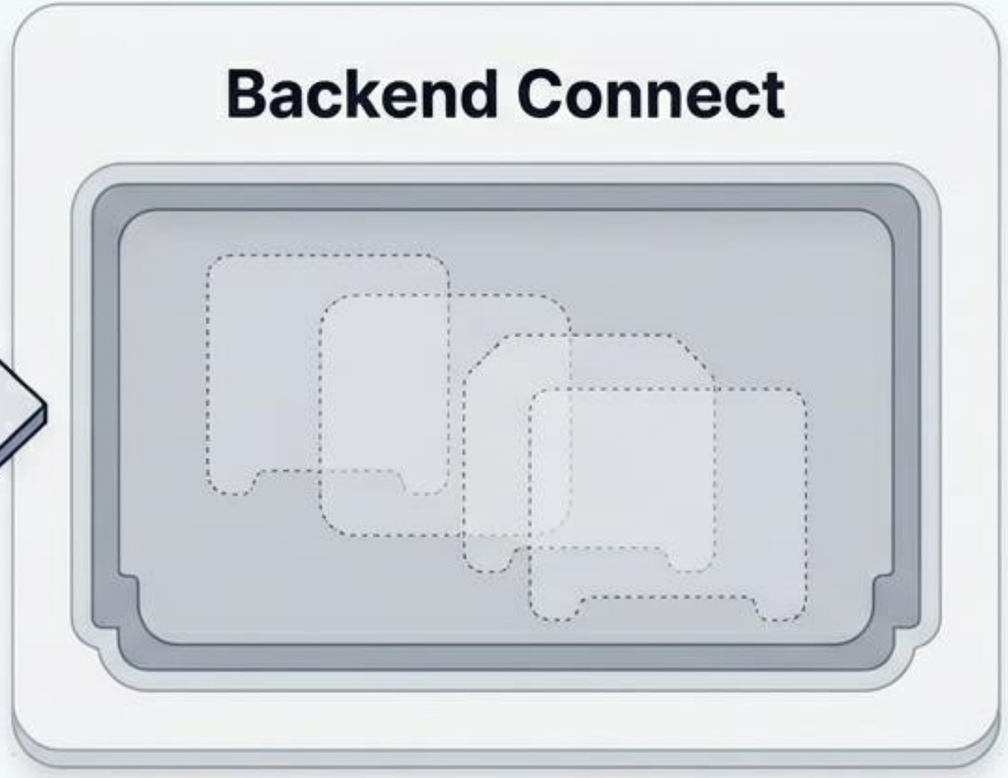
**Realtime BaaS**

Cloud Hosted & Instant

Every paradigm requires a completely different mental model for how data is shaped, stored, and retrieved.

NotebookLM

# The Anchor App: Vote for your favorite game




**Same UI, different database implementations.**

NotebookLM

# Architecture 1: SQLite

**What is SQLite?** A self-contained, serverless, zero-configuration, transactional SQL database engine **perfectly suited for local development and learning.**

**Project Files**

- astro-project
  - src
    - pages
    - components
    - database.sqlite

The entire database is just one local file.

| id | title | votes |
|---|---|---|
| 1 | Pac-Man | 42 |
| 2 | Tetris | 38 |

**Local DB + SQL basics.**
Perfect for learning without external dependencies.

NotebookLM

# Executing Raw SQL Operations

'Arcade Ranker'

Pac-Man 42 Vote

Tetris 38 Vote

**database.js** in Roboto

```
// 1. Setup Table
db.exec('CREATE TABLE games (id INTEGER
   PRIMARY KEY, title TEXT, votes INTEGER)');

// 2. Insert Data
db.prepare('INSERT INTO games (title, votes)
   VALUES (?, ?)').run('Pac-Man', 0);

// 3. Increment Vote
db.prepare('UPDATE games SET votes = votes + 1
   WHERE id = ?').run(gameId);
```

NotebookLM

# Architecture 2: PostgreSQL + Prisma

What is Prisma? An Object-Relational Mapper (ORM). It bridges the gap between object-oriented JavaScript and the rigid tables of a PostgreSQL database.

**JavaScript**

```
{
  id: 1,
  name:
  'JS Object'
}
```

**The Translator**

**Prisma Engine**
(Translator)

**SQL Tables**

| id | name | votes |
|---|---|---|
| 1 | JS Object | 0 |
| | | |
| | | |
| | | |

**schema.prisma**

```
model Game {
  id    Int    @id @default(autoincrement())
  title String
  votes Int    @default(0)
}
```

Structured data + production patterns. **Type safety from database to frontend.**

NotebookLM

# Moving to Production: Typed Queries

**Raw SQL (String based)**

```
const query = `UPDATE games SET
  votes = votes + 1
  WHERE id = ${id}`;

await db.query(query);
```

Prone to typos, vulnerable to injection, no autocomplete.

**Prisma (Object based)**

```
await prisma.game.update({
  where: { id: gameId },
  data: { votes: { increment: 1 } }
});
```

Fully typed, autocomplete enabled, structurally safe.

NotebookLM

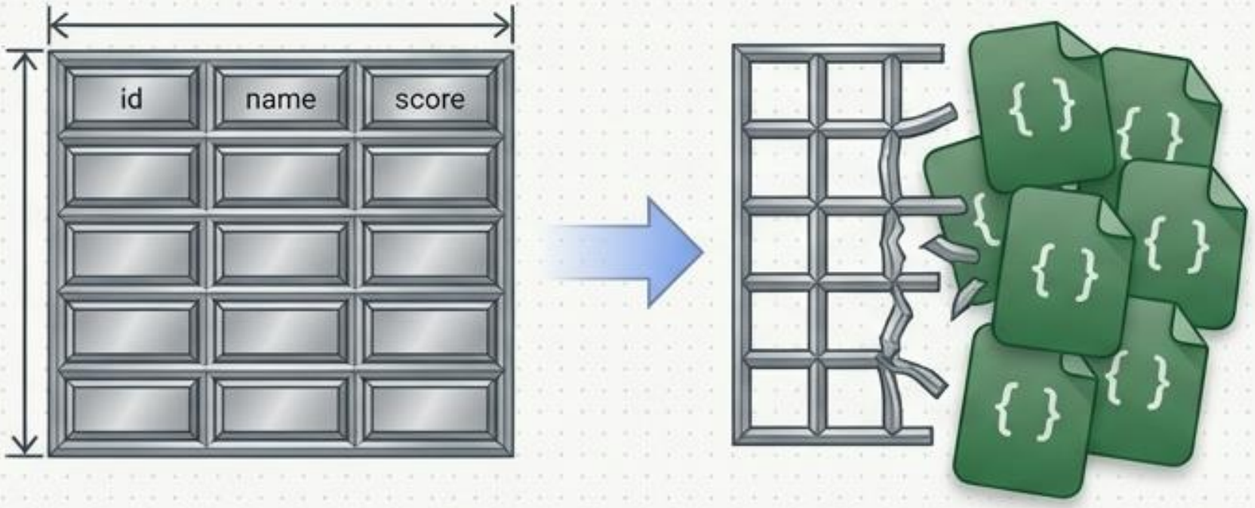
# Architecture 3: MongoDB (NoSQL)

**What is NoSQL?**
A departure from rigid relational tables. Data is stored in flexible, JSON-like documents. Schemas can evolve dynamically without massive migrations.

MongoDB Document (BSON)

```
{
  _id: '64a1b2c3',
  title: 'Pac-Man',
  votes: 42,
  metadata: {
    genre: 'Arcade',
    year: 1980
  }
}
```

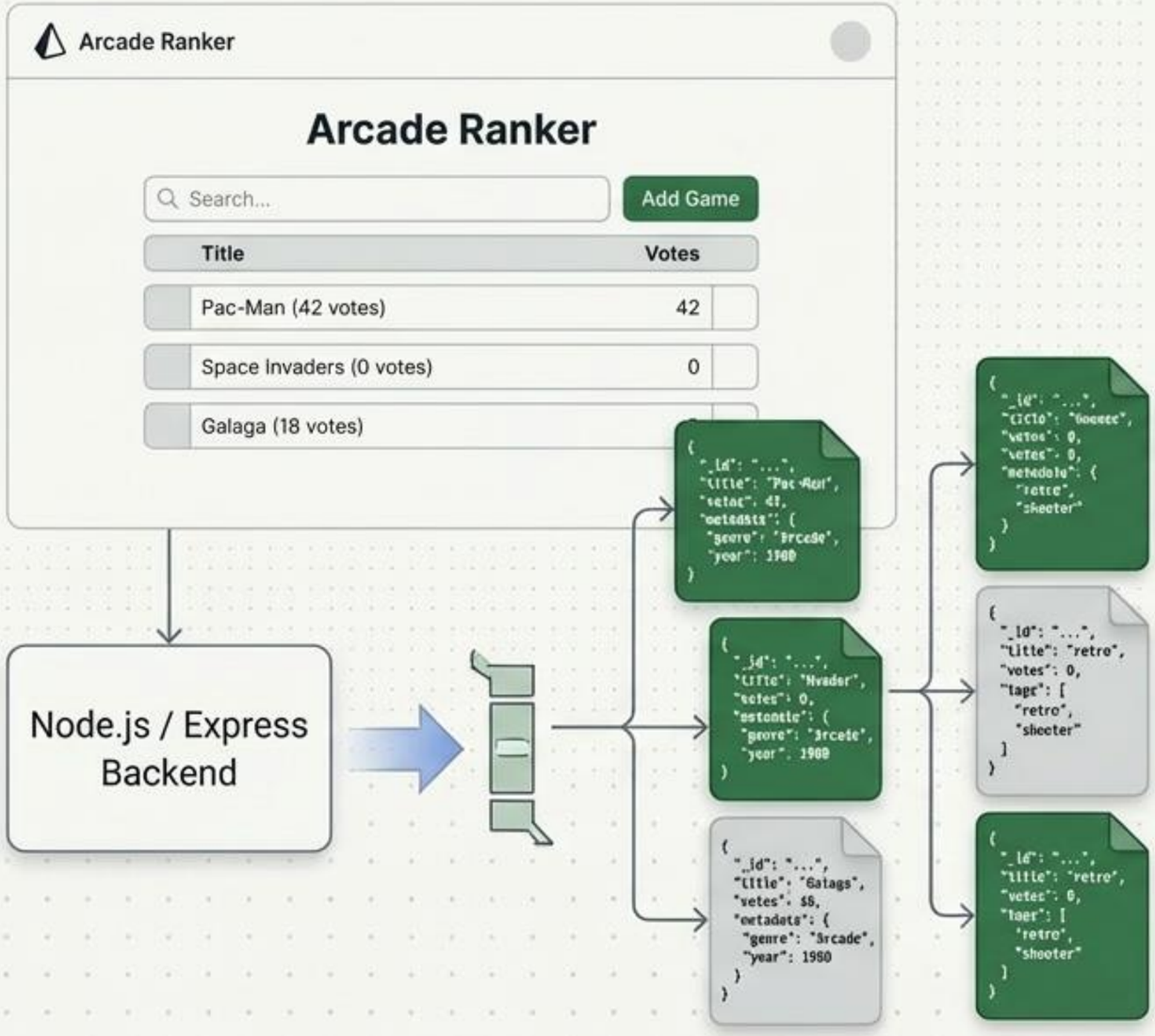
**Schema vs Schemaless**




**Flexible data modeling.**
No strict schema, no joins, just documents.

NotebookLM

# Interacting with Document Stores




mongoose.js

```
// 1. Fetch all documents
const games = await Game.find();

// 2. Add a new document (flexible schema)
await Game.create({
  title: 'Space Invaders',
  votes: 0,
  tags: ['retro', 'shooter']
});

// 3. Update document directly
await Game.findByIdAndUpdate(
  gameId,
  { $inc: { votes: 1 } }
);
```

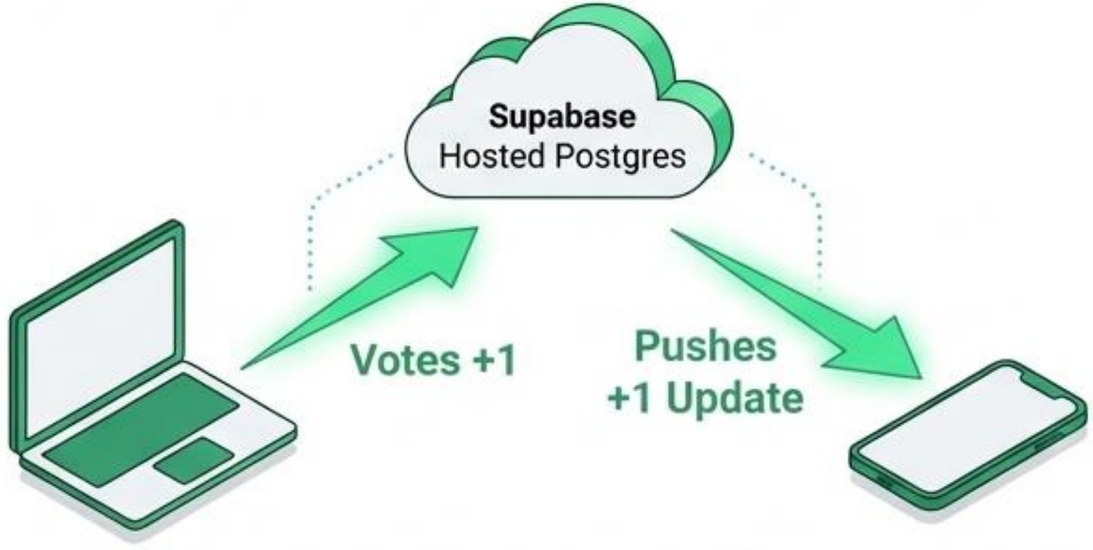
Mongoose ODM for structured interaction.

NotebookLM

# Architecture 4: Supabase (Realtime BaaS)

**What is Supabase?**

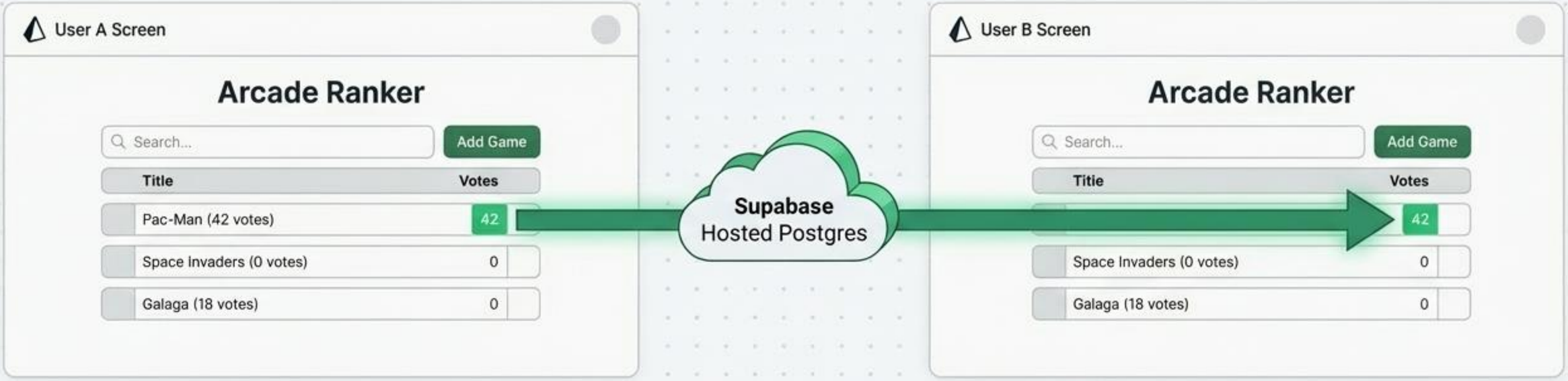
An open-source Backend-as-a-Service (BaaS) providing a hosted Postgres database combined with auto-generated APIs and instant realtime WebSocket subscriptions.



**One student votes -> everyone sees it instantly. The fastest way to build modern collaborative apps.**

NotebookLM

# Subscribing to Live Database Changes




```
// Listen to live database changes from the frontend
supabase
  .channel('games-channel')
  .on('postgres___changes', { event: 'UPDATE', schema: 'public', table: 'games'}, (payload) => {
      updateUIVoteCount(payload.new.id, payload.new.votes);
  })
  .subscribe();
```

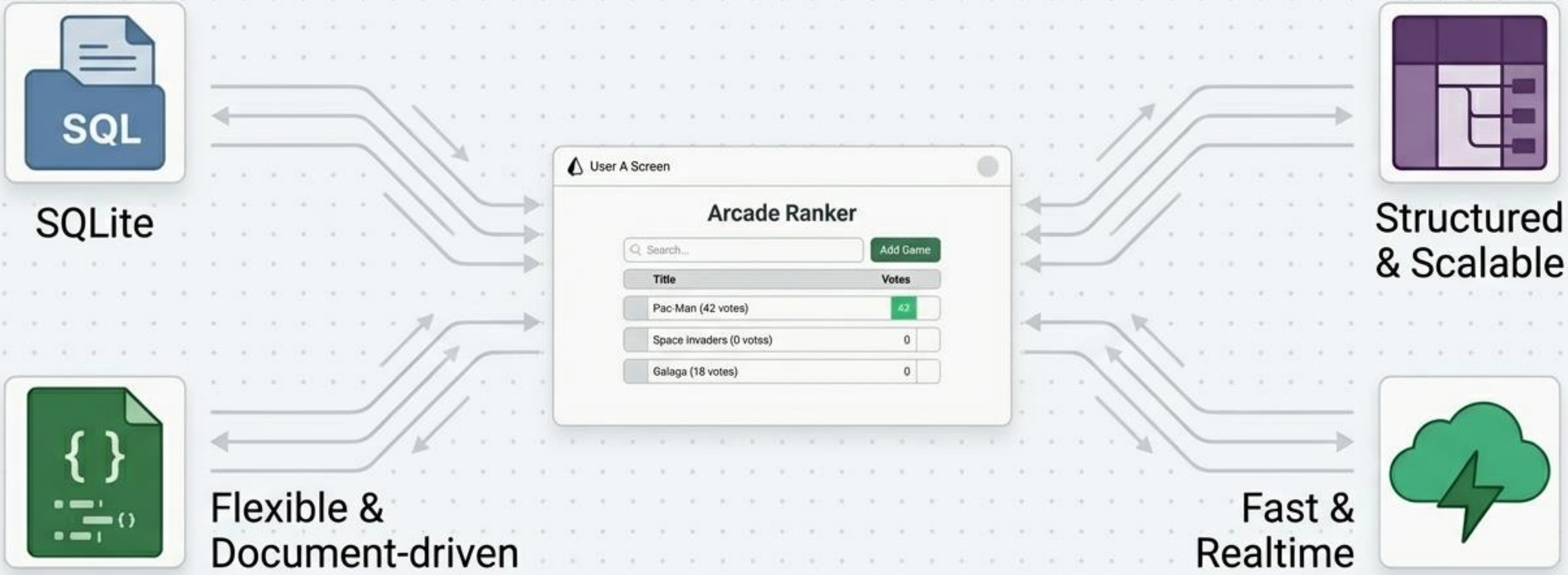
NotebookLM

# Which Database Should I Choose?

| Database | Structure | Setup Friction | Realtime | Best Use Case |
|---|---|---|---|---|
| SQLite | Relational File | Low friction | ✗ | Learning & Local |
| PostgreSQL + Prisma | Relational Server | High friction | ✗ | Enterprise Scale |
| MongoDB | Document JSON | Medium friction | ✗ | Rapid Prototyping |
| Supabase | Relational + APIs | Medium friction | ✓ | Live Collaboration |

NotebookLM

# The Architecture Defines the Approach




Same app, different database = different mental models.

NotebookLM